



OBJECT DESIGN DOCUMENT

Progetto Ingegneria del Software a.a. 2025/2026



28 MAGGIO 2026

- GABRIELE ANDREA TRINCERI
- SANTINO MILANO
- DAVIDE CIACCIO
- GIUSEPPE MANDRACCHIA

Sommario

1 Introduzione	2
1.1 Compromessi e scelte nella progettazione	2
2 Packages	4
2.1 Package Entity (Model)	4
2.2 Package Control (Controller)	5
2.3 Package Boundary	9
2.4 Package View (Risorse FXML)	10
2.5 Package Utility, Server e TextMessage	12

1 Introduzione

1.1 Compromessi e scelte nella progettazione

Per la realizzazione del sistema **ShareRoomAfam** si è scelto un approccio modulare orientato agli oggetti. Il sistema segue il pattern architetturale **MVC (Model-View-Controller)**, che permette di separare nettamente la logica di business e l'accesso ai dati (Model) dalla presentazione visiva (View) e dalla gestione degli eventi generati dall'utente (Controller). Questa scelta garantisce una maggiore manutenibilità del codice e semplifica le fasi di test e debugging.

L'intero progetto è stato sviluppato in linguaggio **Java (versione 17)**, avvalendosi di **Maven** come strumento principale per il "build automation" e la gestione delle dipendenze esterne. A livello architetturale, il codice sorgente è stato strutturato sfruttando il **Java Platform Module System (JPMS)** tramite la definizione del file **module-info.java**. Questa scelta garantisce un forte incapsulamento del codice, definendo confini rigorosi tra i vari package: il sistema dichiara esplicitamente i moduli nativi ed esterni richiesti per l'esecuzione e apre alle librerie di reflection (come JavaFX) solo gli strati necessari (boundary e control), mantenendo così blindata la logica di business.

Per quanto concerne l'interfaccia grafica utente (GUI), si è scelto di utilizzare il framework **JavaFX**. L'utilizzo di questo framework, unito alla definizione delle interfacce tramite file FXML, permette di creare schermate moderne e reattive, disaccoppiando completamente il design dell'interfaccia dal codice Java che ne gestisce i comportamenti.

Per l'implementazione delle funzionalità di base, l'accesso ai dati e la costruzione dell'interfaccia, il modulo principale richiede le seguenti dipendenze (dichiarate tra pom.xml e module-info.java):

- **javafx.controls / javafx.fxml (17.0.6)**: Forniscono i componenti UI standard di JavaFX (es. pulsanti, tabelle, campi di testo) e permettono di definire l'interfaccia utente in modo dichiarativo tramite un linguaggio di markup basato su XML, facilitando la separazione tra la logica (Controller) e la presentazione (View).

- **java.sql e mysql-connector-j (8.3.0)**: Il sistema utilizza il modulo nativo Java per la gestione dei database relazionali, affiancato dal driver JDBC ufficiale per interfacciarsi in modo sicuro ed efficiente con il database MySQL, scelto per la persistenza delle entità.

- **jakarta.mail (2.0.1)**: Libreria utilizzata per la gestione e l'invio automatizzato di messaggi di posta elettronica. Nello specifico, è fondamentale per gestire i flussi di sicurezza, come l'implementazione dell'Autenticazione a Due Fattori (2FA) tramite email.

- **jdk.httpserver**: Modulo nativo Java utilizzato per implementare un server HTTP leggero (gestito tramite il componente interno WebServerManager), utile per esporre servizi di rete senza dover integrare container web più pesanti.

- **javafx-maven-plugin (0.0.8)**: Plugin Maven che facilita il ciclo di vita dell'applicazione, permettendo la compilazione, l'esecuzione e il packaging nativo dell'applicativo JavaFX direttamente da riga di comando.

2 Packages

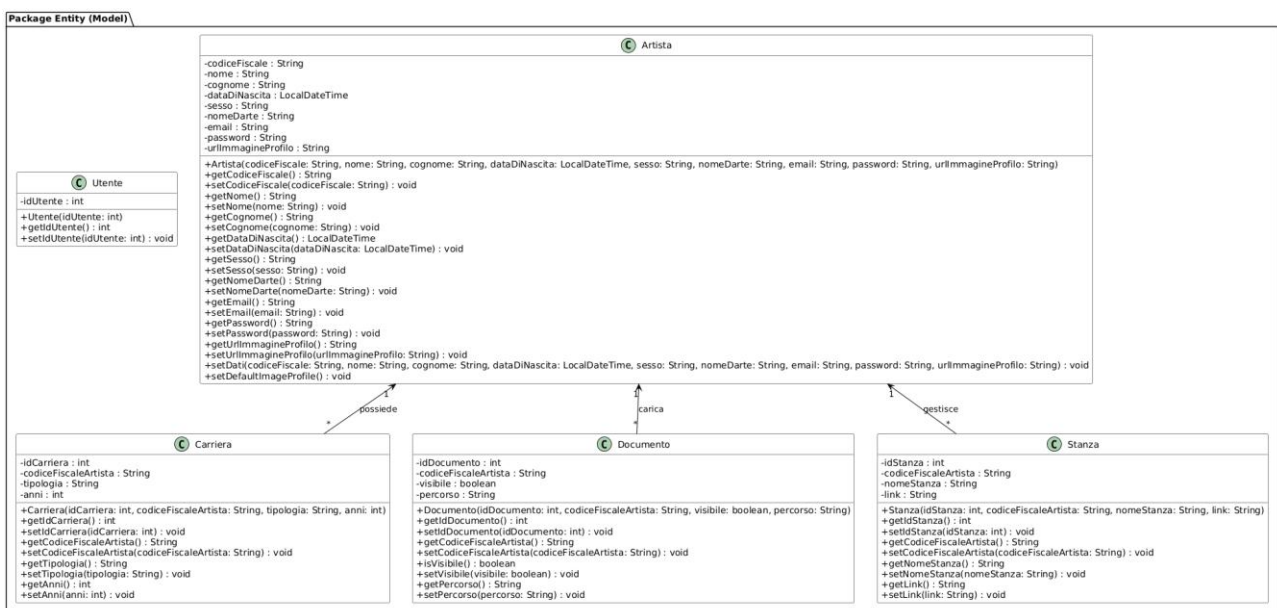
Per un'organizzazione ordinata e modulare durante lo sviluppo del software, si è scelto di dividere le componenti in diversi packages seguendo rigorosamente l'architettura **MVC (Model-View-Controller)** abbinata al pattern **BCE (Boundary-Control-Entity)**. Il software è stato suddiviso in livelli logici per separare la gestione dei dati, la logica di business, l'interfacciamento con i sistemi esterni e la presentazione grafica all'utente.

A seguire, l'elenco dei package principali e le relative classi:

2.1 Package Entity (Model)

Questo package contiene le classi di dominio che rappresentano le entità fondamentali del sistema **ShareRoomAfam** e il cui stato deve essere reso persistente.

Dettaglio Classi:

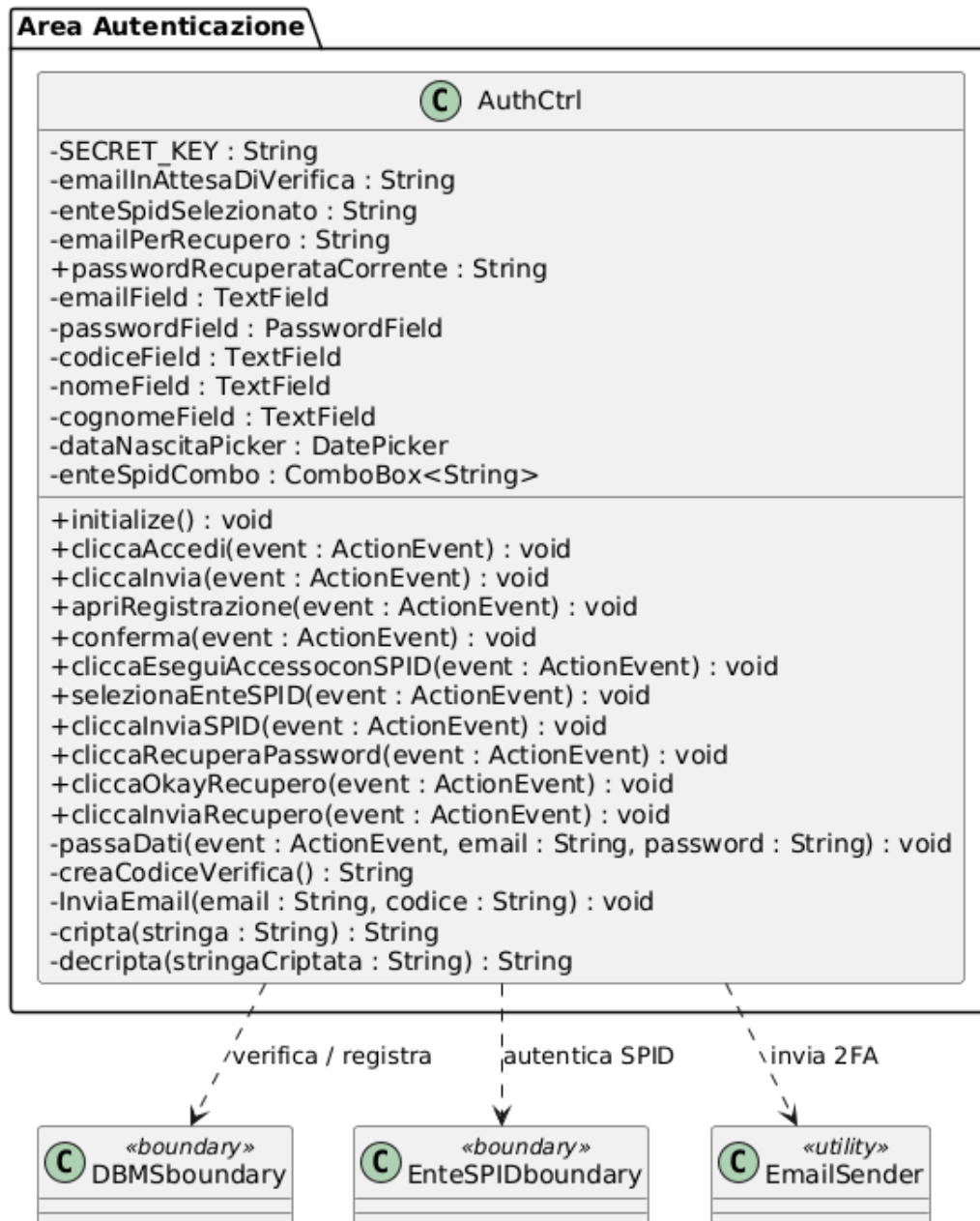


2.2 Package Control (Controller)

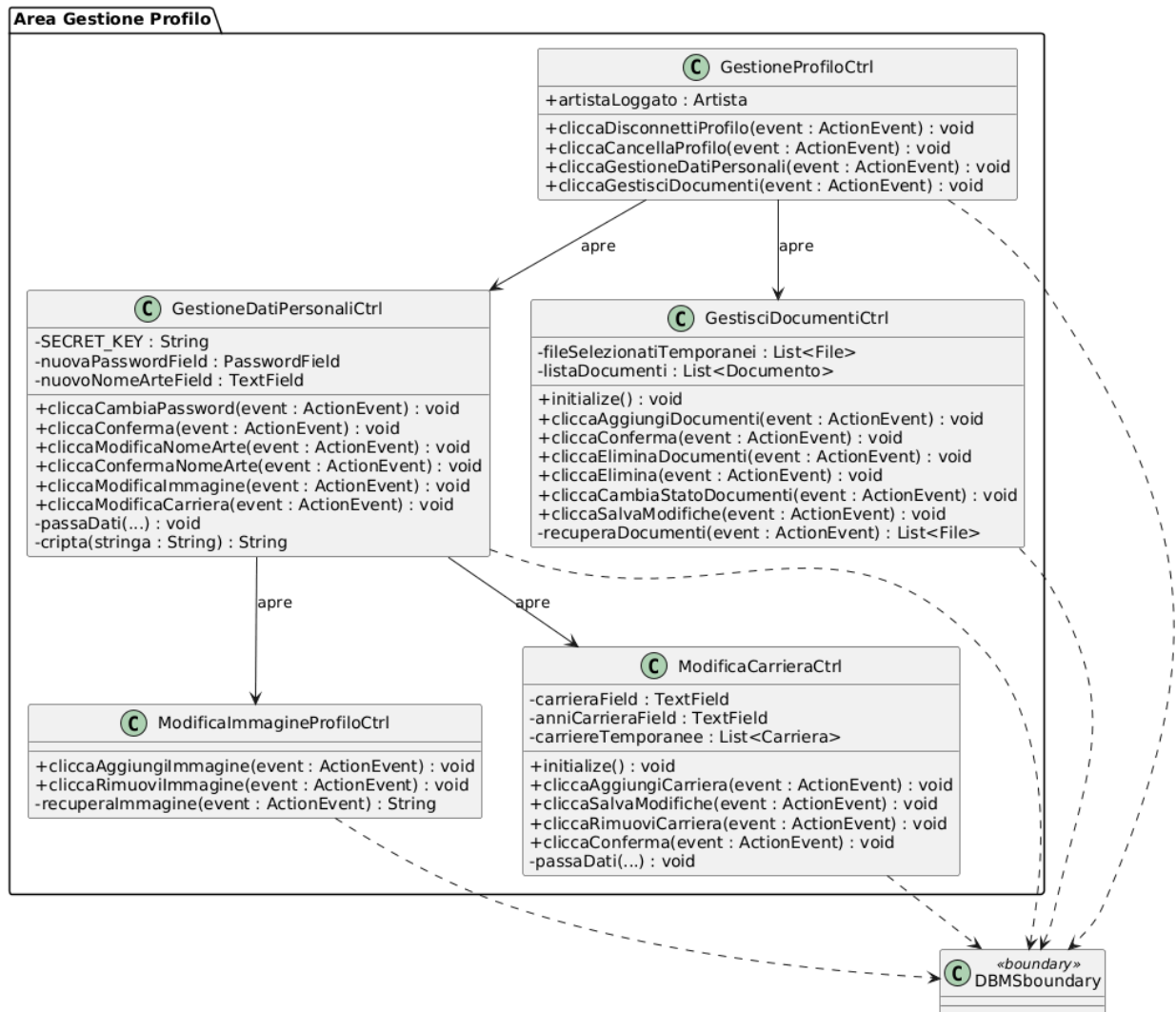
Il package control ospita i gestori della logica di business. Queste classi fungono da intermediari tra l'interfaccia grafica (View) e i dati (Entity), elaborando gli input e applicando le regole del sistema.

Dettaglio Classi:

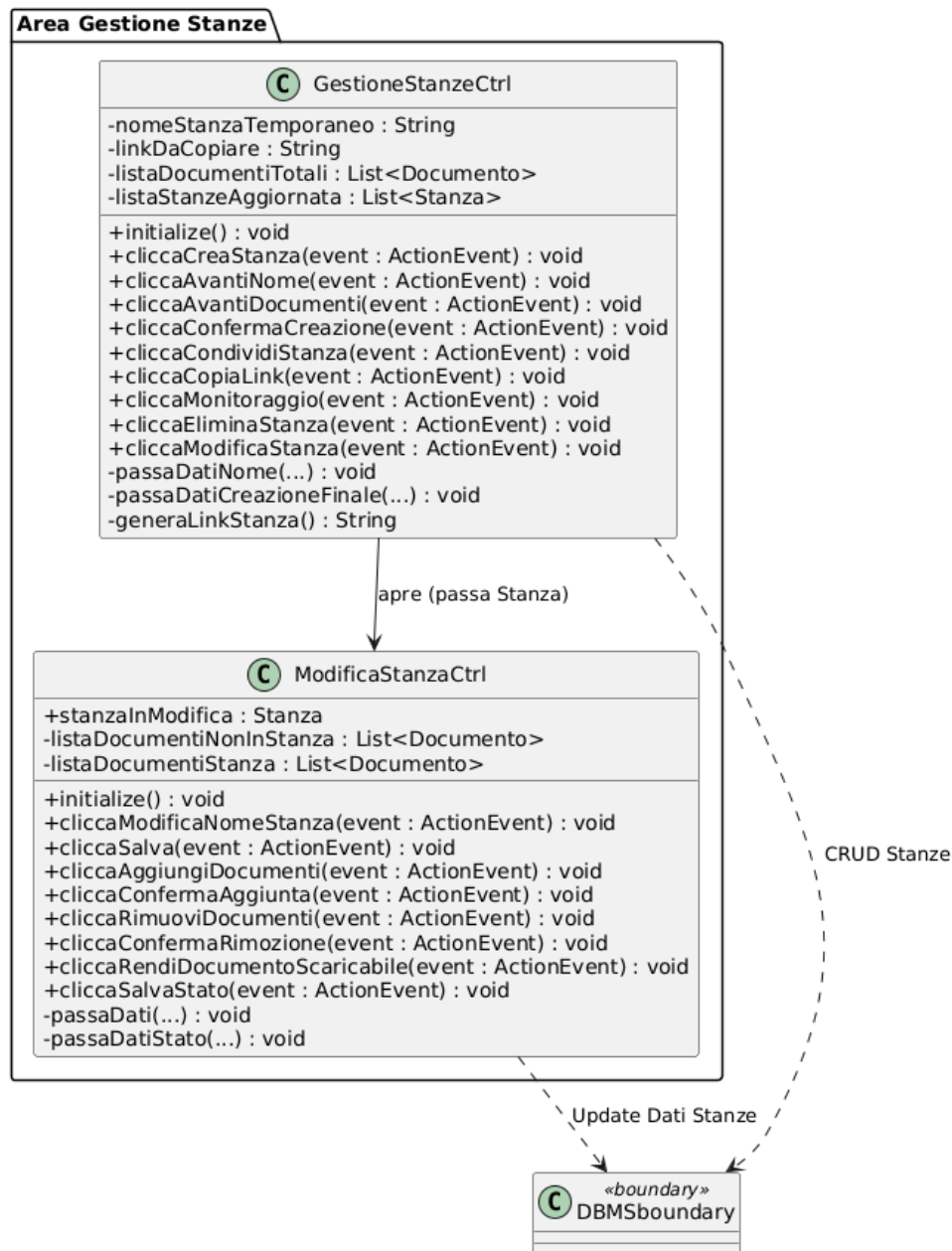
Area autenticazione



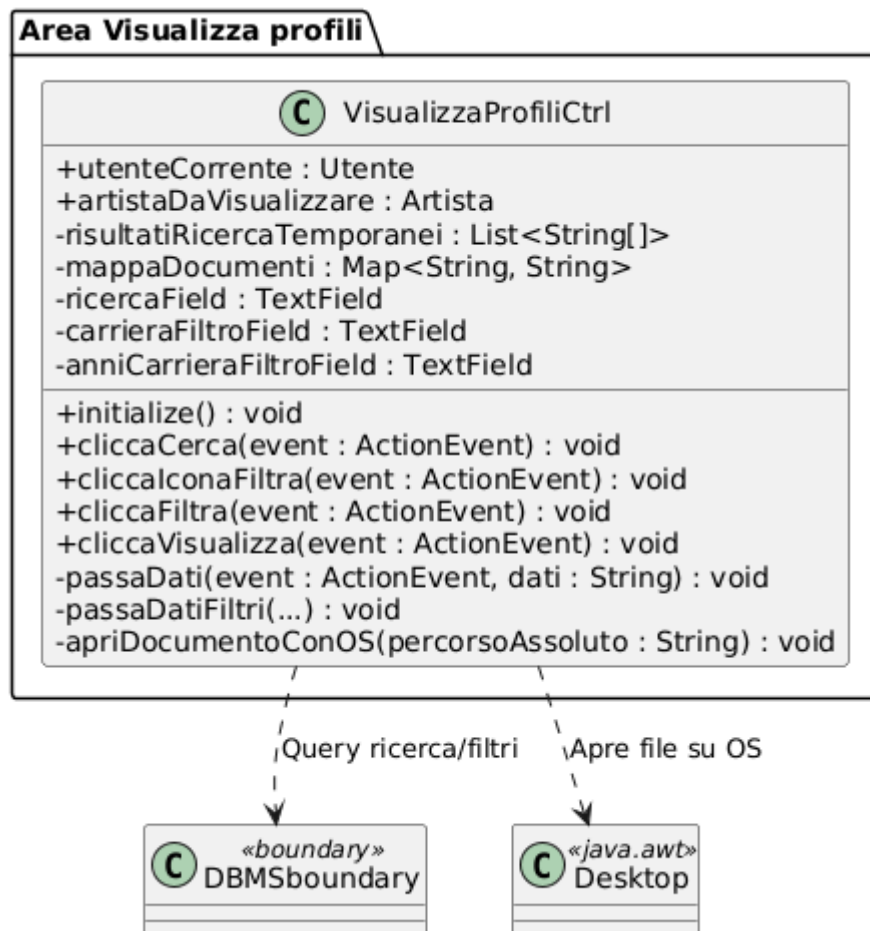
Area gestione profilo



Area gestione stanze



Area visualizza profili



2.3 Package Boundary

Il package boundary contiene le classi deputate all'interfacciamento con i sistemi e i servizi esterni all'applicazione, isolando così il resto del codice da dettagli implementativi di basso livello.

Dettaglio Classi:

DBMSboundary: Gestisce le connessioni dirette e le query al database MySQL (definito tramite lo script schema.sql). Traduce gli oggetti Entity in record relazionali e viceversa.

EnteSPIDboundary: Componente adibito all'integrazione e alla comunicazione con l'identity provider per l'accesso tramite SPID.

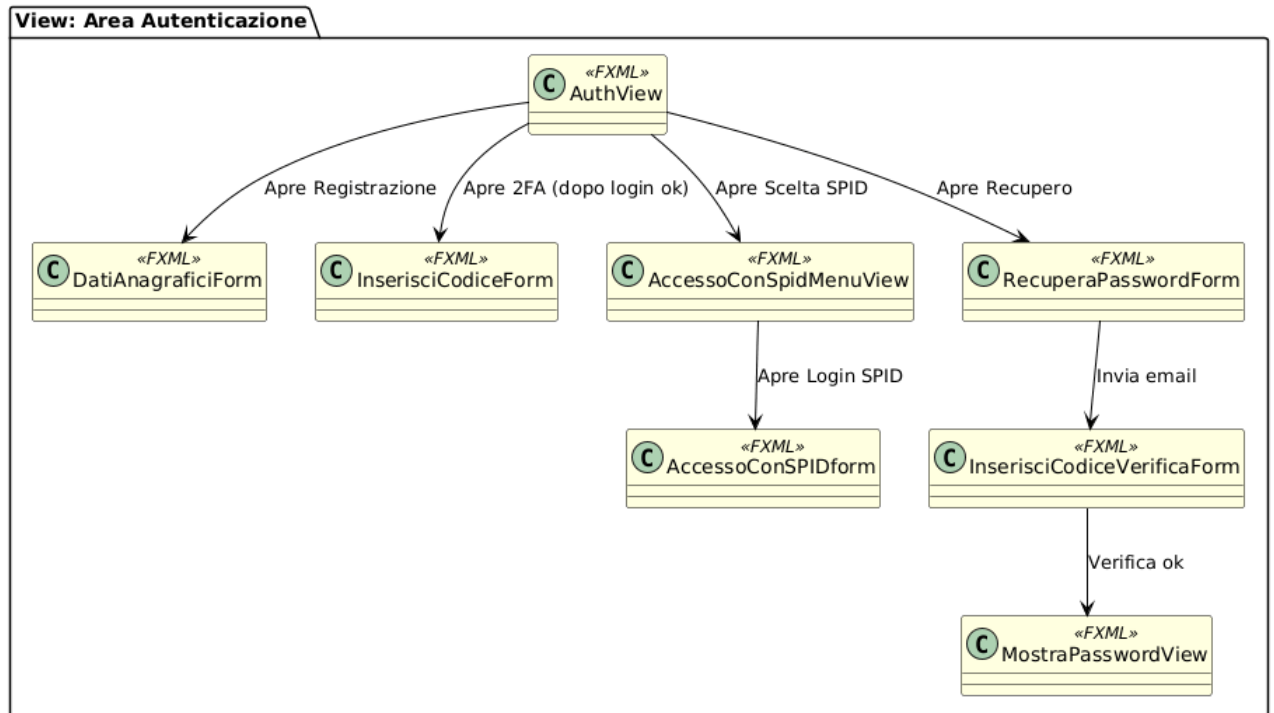


2.4 Package View (Risorse FXML)

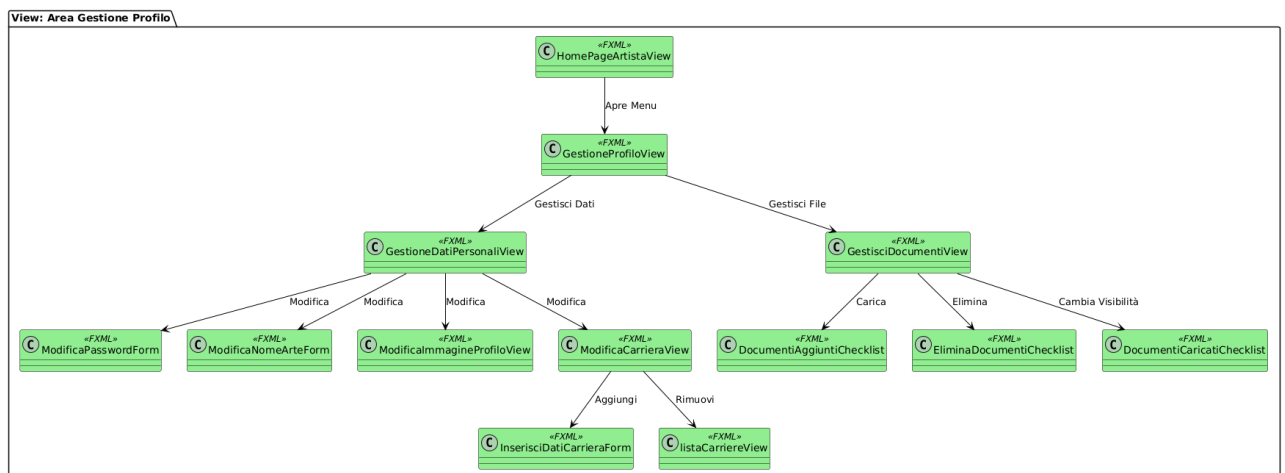
Non essendo basata su pagine web ma su JavaFX, la UI è contenuta all'interno della directory resources/com.shareroomafam.view. Qui sono presenti i file di markup dichiarativo FXML che definiscono la struttura visiva dell'applicazione, abbinati al foglio di stile style.css.

Dettaglio Schermate Principali:

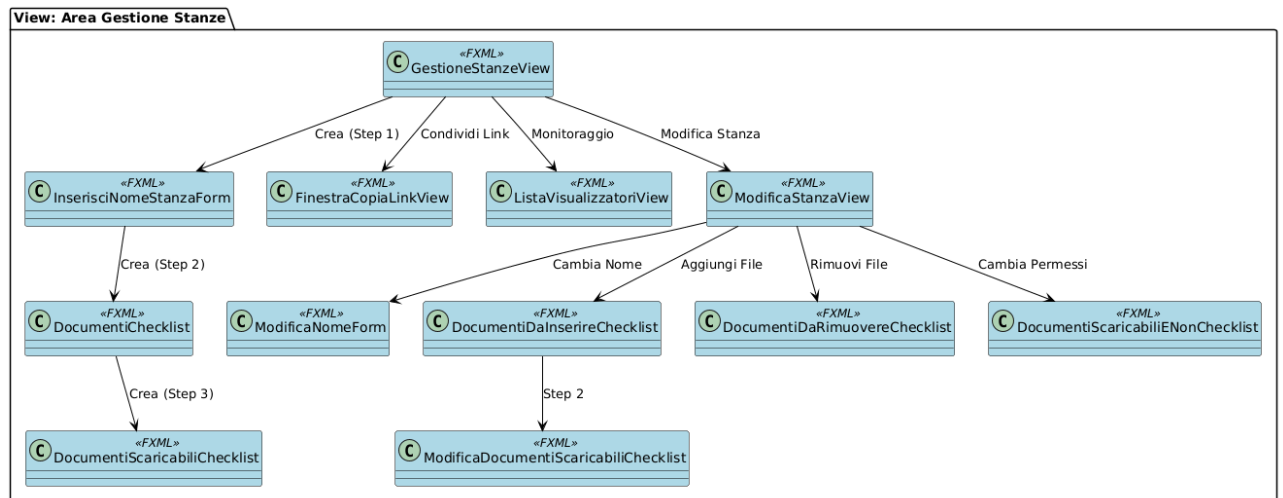
Area autenticazione



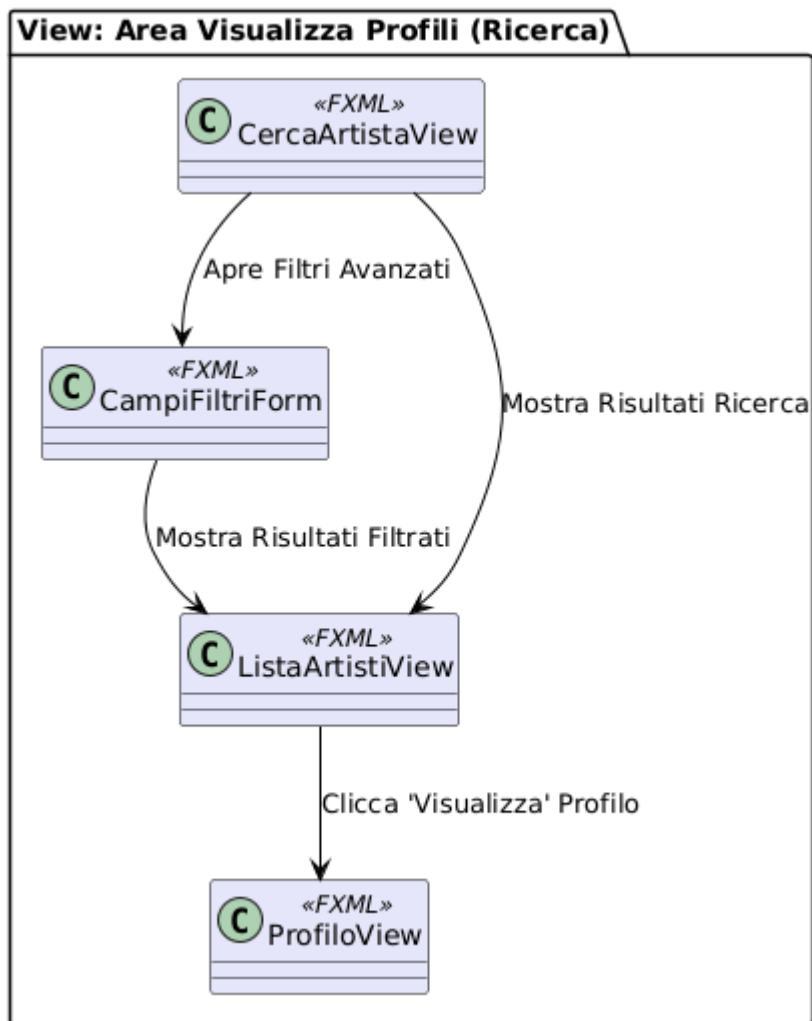
Area gestione profilo



Area gestione stanze



Area visualizza profilo



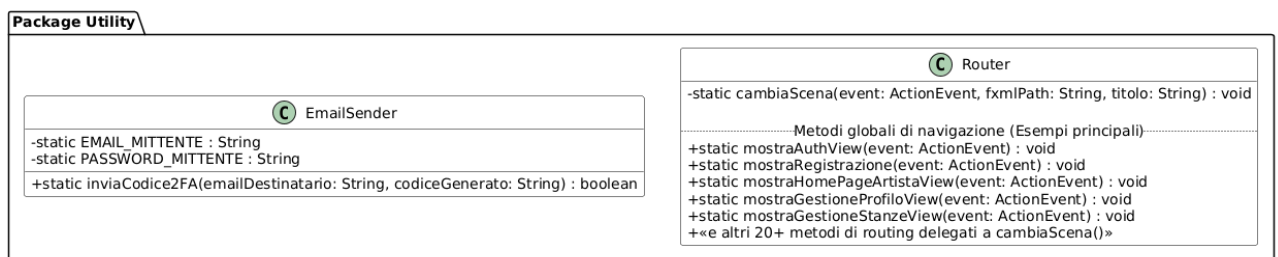
2.5 Package Utility, Server e TextMessage

Moduli di supporto per l'intera architettura.

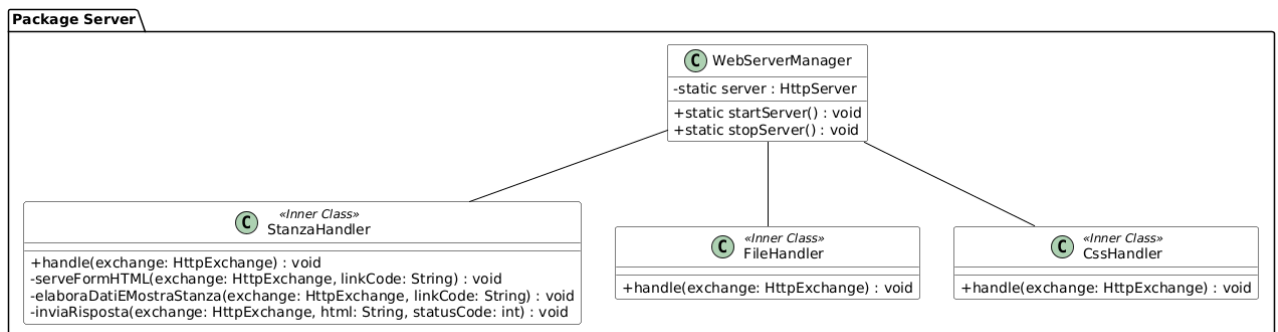
Dettaglio Classi:

EmailSender: Classe di utilità (utility) che implementa la libreria Jakarta Mail per l'invio automatizzato di email (es. per conferme, recupero password e 2FA).

Router: Gestisce la navigazione interna dell'applicativo, caricando e scambiando i file FXML sullo stage principale di JavaFX.



WebServerManager: Componente lato server per eventuali esposizioni di servizi o comunicazioni di rete.



ConfirmText / ErrorText / SuccessfulText: Classi nel package textmessage per la centralizzazione e standardizzazione dei messaggi di feedback testuali forniti all'utente (errori, successi, conferme).

